CS7367 MACHINE VISION ASSIGNMENT 1

Sreehitha Korrapati-6/10/2026
KSU ID: 001228797

ABSTRACT:

In this assignment we work on to see how image sampling and gray-level quantization affect image quality. A 1024×1024 grayscale rose image is used as the original image. Now, to examine the effect of sampling, the image is first reduced to several lower resolutions (512×512, 256×256, 128×128, 64×64, and 32×32) and then resized back to 1024×1024. As the resolution decreases, the image loses fine details and appears more pixelated when enlarged. For gray-level quantization, the number of intensity levels was reduced from 256 to 128, 64, 32, 16, 8, 4, and 2. With fewer gray levels, the image becomes less smooth and noticeable banding appears, especially at very low levels. These results show that both spatial resolution and gray-level resolution have a clear impact on image quality.

TEST RESULTS:

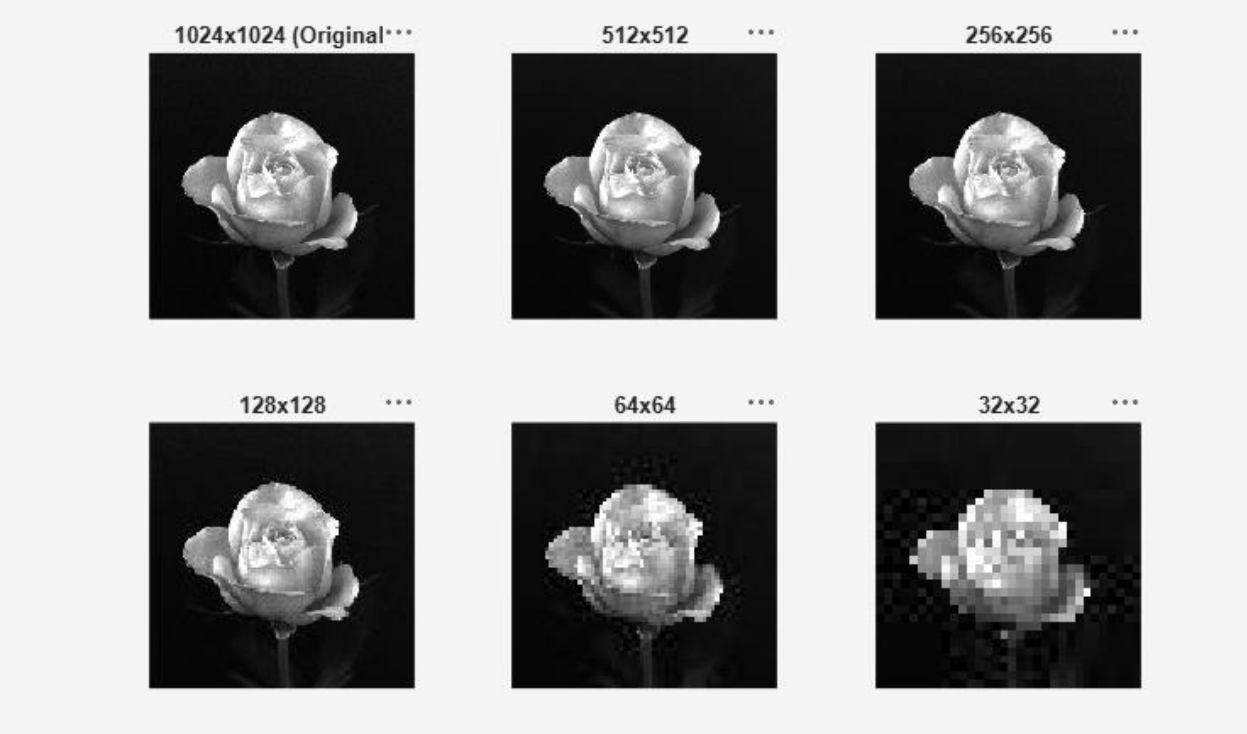

Fig1: Downsampling: The original 1024×1024 image was down sampled using nearest-neighbor interpolation. Quality is acceptable at 512×512 and 256×256, but block artifacts become clearly visible at 64×64 and 32×32.

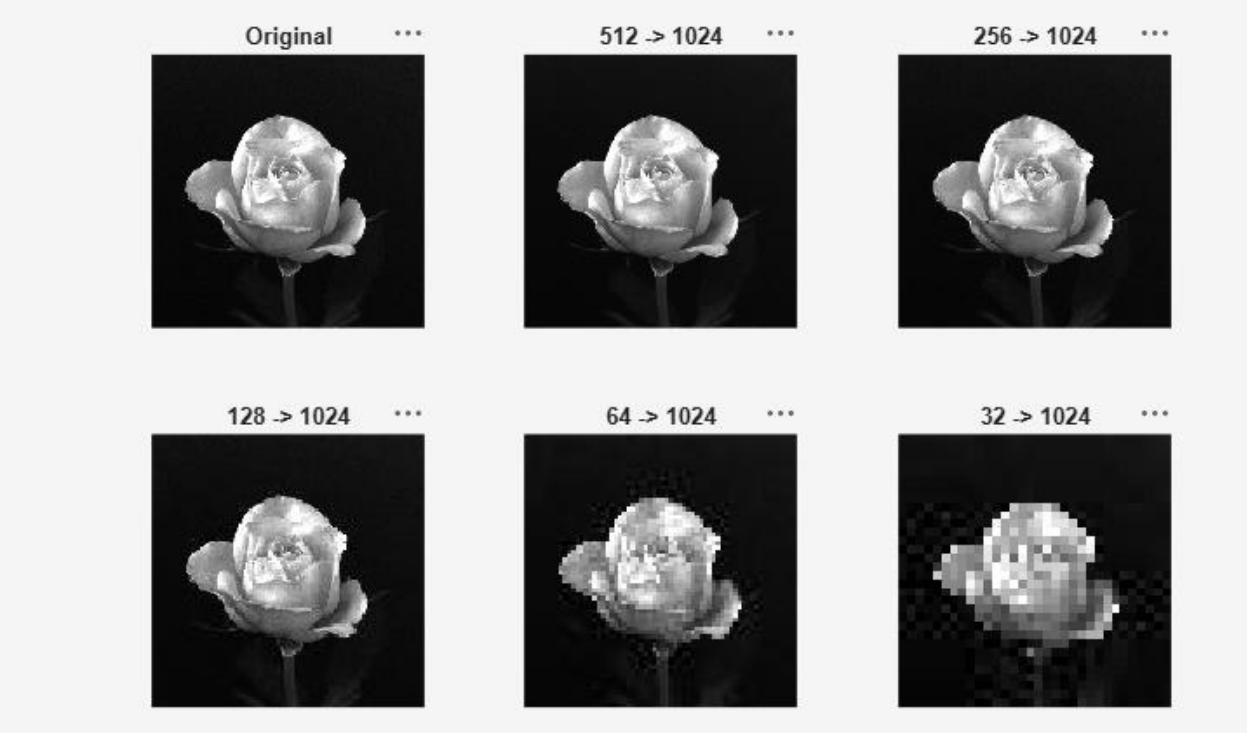

Fig 2: Upsampling: Back to 1024×1024 Each downsampled image was upsampled back to 1024×1024 using the same nearest-neighbor method. Lost detail cannot be recovered and images downsampled to lower resolutions also show clear block artifacts after reconstruction

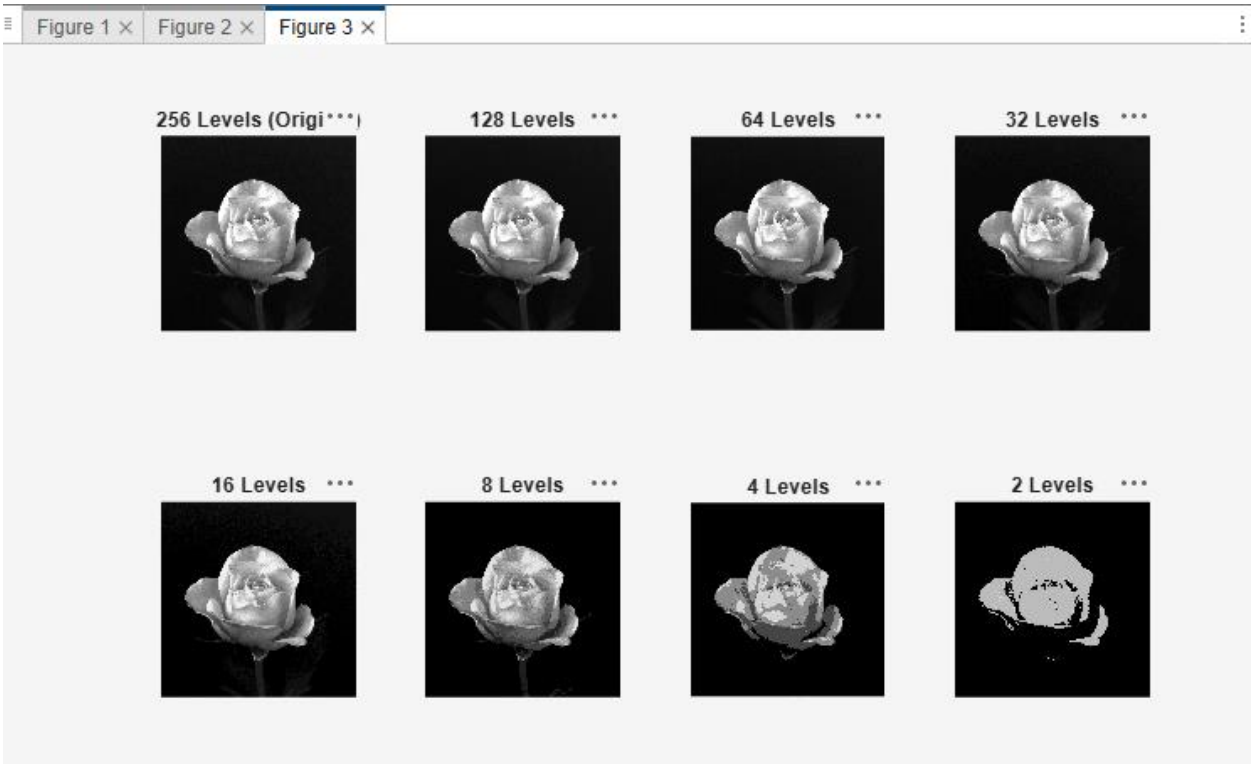

Fig3: Gray-Level Quantization: The grayscale image was quantized by reducing the number of intensity levels. At 128 and 64 levels the result looks nearly the same as the original. Contouring becomes visible at 16 levels and below, and at 2 levels the image essentially becomes binary.

DISCUSSION:

Working on this assignment helped me understand how digital images are stored and how sampling and quantization affect quality. The main lesson was that information lost during downsampling cannot be recovered by upsampling. I also noticed that gray-level reduction below 16 levels produces obvious contouring in smooth areas. Implementing everything with loops instead of built-in functions made it easier to understand exactly how each pixel value is computed.

CODE:

```
clear;
clc;
close all;

%% Downsampling
img = imread('rose.jpg');
if size(img, 3) == 3
   img = rgb2gray(img);
end

[r, c] = size(img);

sizes = [512 256 128 64 32];

for s = 1:length(sizes)

   newSize = sizes(s);

   output = zeros(newSize, newSize, 'uint8');
   factor = r / newSize;

   for i = 1:newSize
      for j = 1:newSize

         old_i = floor((i-1) * factor) + 1;
         old_j = floor((j-1) * factor) + 1;

         output(i, j) = img(old_i, old_j);

      end
   end

   filename = sprintf('rose_%d.jpg', newSize);
   imwrite(output, filename);

end

figure;

subplot(2, 3, 1);
imshow(imread('rose.jpg'), []);
```

```
title('1024x1024 (Original)');

subplot(2, 3, 2);
imshow(imread('rose_512.jpg'), []);
title('512x512');

subplot(2, 3, 3);
imshow(imread('rose_256.jpg'), []);
title('256x256');

subplot(2, 3, 4);
imshow(imread('rose_128.jpg'), []);
title('128x128');

subplot(2, 3, 5);
imshow(imread('rose_64.jpg'), []);
title('64x64');

subplot(2, 3, 6);
imshow(imread('rose_32.jpg'), []);
title('32x32');

saveas(gcf, 'downsampling_comparison.png');

%% Upsampling

sizes = [512 256 128 64 32];

for s = 1:length(sizes)

    smallSize = sizes(s);

    small = imread(sprintf('rose_%d.jpg', smallSize));

    if size(small, 3) == 3
        small = rgb2gray(small);
    end

    [rs, cs] = size(small);

    output = zeros(1024, 1024, 'uint8');

    factor = rs / 1024;

    for i = 1:1024
        for j = 1:1024

            old_i = floor((i-1) * factor) + 1;
            old_j = floor((j-1) * factor) + 1;

            if old_i > rs
                old_i = rs;
            end
            if old_j > cs
                old_j = cs;
            end
```

```
            output(i, j) = small(old_i, old_j);

        end
    end

    filename = sprintf('rose_%d_to_1024.jpg', smallSize);
    imwrite(output, filename);

end

figure;

subplot(2, 3, 1);
imshow(imread('rose.jpg'), []);
title('Original');

subplot(2, 3, 2);
imshow(imread('rose_512_to_1024.jpg'), []);
title('512 -> 1024');

subplot(2, 3, 3);
imshow(imread('rose_256_to_1024.jpg'), []);
title('256 -> 1024');

subplot(2, 3, 4);
imshow(imread('rose_128_to_1024.jpg'), []);
title('128 -> 1024');

subplot(2, 3, 5);
imshow(imread('rose_64_to_1024.jpg'), []);
title('64 -> 1024');

subplot(2, 3, 6);
imshow(imread('rose_32_to_1024.jpg'), []);
title('32 -> 1024');

saveas(gcf, 'sampling_comparison.png');

%% Gray-level quantization
gray = imread('rose.jpg');
if size(gray, 3) == 3
    gray = rgb2gray(gray);
end

levelsList = [128 64 32 16 8 4 2];

for k = 1:length(levelsList)

    levels = levelsList(k);

    step = 256 / levels;

    output = zeros(size(gray), 'uint8');

    for i = 1:size(gray, 1)
```

```
        for j = 1:size(gray, 2)

            pixel = double(gray(i, j));
            output(i, j) = uint8(floor(pixel / step) * step);

        end
    end

    filename = sprintf('gray_%dlevels.jpg', levels);
    imwrite(output, filename);

end

figure;

subplot(2, 4, 1);
imshow(gray, []);
title('256 Levels (Original)');

subplot(2, 4, 2);
imshow(imread('gray_128levels.jpg'), []);
title('128 Levels');

subplot(2, 4, 3);
imshow(imread('gray_64levels.jpg'), []);
title('64 Levels');

subplot(2, 4, 4);
imshow(imread('gray_32levels.jpg'), []);
title('32 Levels');

subplot(2, 4, 5);
imshow(imread('gray_16levels.jpg'), []);
title('16 Levels');

subplot(2, 4, 6);
imshow(imread('gray_8levels.jpg'), []);
title('8 Levels');

subplot(2, 4, 7);
imshow(imread('gray_4levels.jpg'), []);
title('4 Levels');

subplot(2, 4, 8);
imshow(imread('gray_2levels.jpg'), []);
title('2 Levels');

saveas(gcf, 'quantization_comparison.png');
```